

PROGRAMACIÓN 1

Modularización



Universidad Católica de La Plata
Facultad de Ciencias Exactas y Tecnología

Andrés Garavaglia
Antonio Restelli

Contenido

- Programación modular
 - Características
 - Funciones y Procedimientos
 - Parámetros
 - Pasaje de parámetros
 - Ámbito de las variables

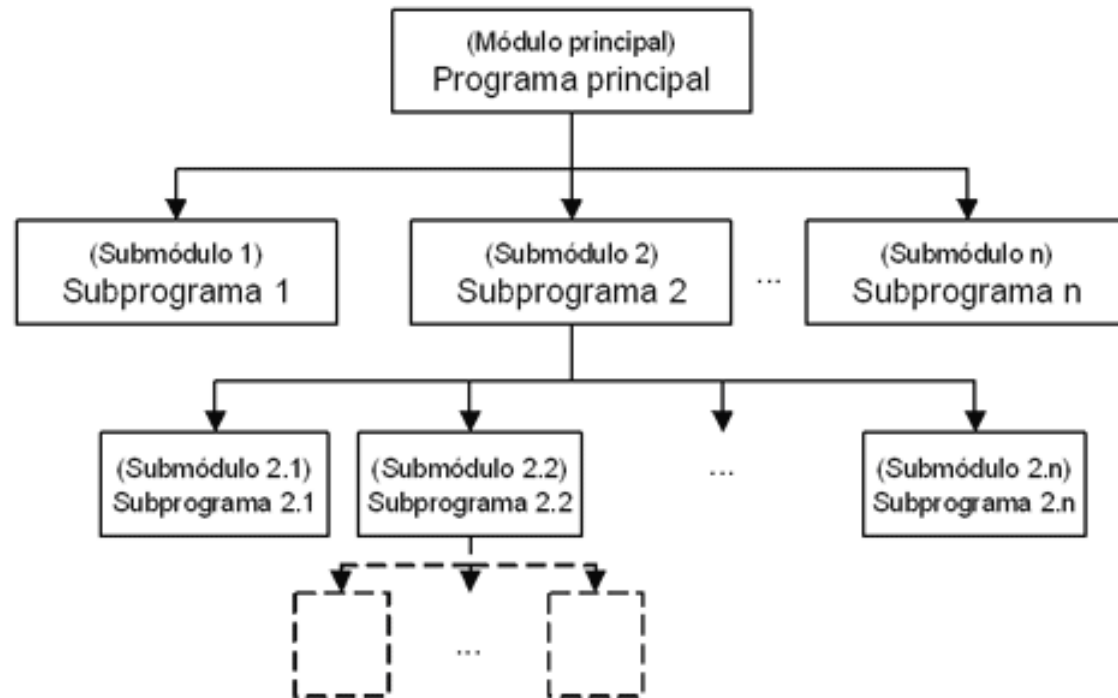
Programación Modular



Es un método de diseño flexible para mejorar la productividad de un programa. El programa se divide en módulos, cada uno de los cuales ejecuta una única actividad o tarea y se codifican independientemente de los otros módulos.

- **Funcionamiento:**
 - **Cada programa contiene un módulo denominado programa principal que transfiere el control a los subprogramas. Estos devuelven el control al módulo principal una vez que hayan terminado su tarea.**
 - **Si la tarea asignada al módulo es demasiado compleja deberá romperse en otros módulos mas pequeños.**
 - **Los módulos son independientes. Ningún modulo puede tener acceso directo a otro módulo excepto al módulo que llama y sus submódulos.**
 - **El resultado producido por un módulo puede ser utilizado por otros módulos una vez que se transfiera el control a ellos.**

Programación Modular



- Características:
 - **Facilita el diseño descendente.**
 - **Los módulos se pueden ejecutar mas de una vez en un programa y en diferentes programas. Ahorrando tiempo de programación**
 - **Facilita la división de tareas en un grupo de programación**
 - **Los módulos se pueden comprobar individualmente y estructurarse en librerías.**

Programación Modular

Ejemplo. Realizar un programa que calcule el área de un rectángulo.

Subprograma 1: Leer la altura y base.

Subprograma 2: Calcular el área.

Subprograma 3: Mostrar el resultado.

```
program area;  
var  
base,altura:real;  
resultado:real;  
BEGIN
```

```
  readln(base);  
  readln(altura);
```

S1

```
  resultado:= base x altura;
```

S2

```
  writeln(resultado);
```

S3

```
END.
```

Funciones



Matemáticamente una función es una operación que toma uno o mas valores denominados argumentos y produce un valor denominado resultado. El valor esta asociado a su nombre.

$$f(x) = \frac{x}{1 + x^2}$$

f es el nombre de la función

X el argumento de la función

Ningún valor en el momento de la definición se asocia con X .

Se denomina parámetro formal.

$$f(3) = 0.3$$

$$f(3) = \frac{3}{1 + 9} = \frac{3}{10} = 0.3$$

Para evaluar el valor de f se debe dar un valor real o actual a X.
Con ese valor se puede calcular el resultado.

Funciones

Declaración de Funciones

- Una cabecera con el nombre, los argumentos y el tipo de valor devuelto
- Cuerpo de la función (serie de acciones o instrucciones cuyo proceso dará un resultado que se devolverá al programa o subprograma llamador.

```
Function F (x:real):real;  
begin  
  result X/(1+X*X);  
end;
```

Invocación de Funciones

- Se establece automáticamente una correspondencia entre parametros formales y actuales
- Debe haber el mismo número de parámetros formales que actuales
- Se supone correspondencia uno a uno de izquierda a derecha.

```
Resultado:= F(2);    // 0.4
```

Procedimientos (subrutinas)



Un procedimiento es un subprograma que ejecuta un proceso específico. Ningún valor está asociado con el nombre del procedimiento. Por consiguiente no puede ocurrir en una expresión.

Procedimiento VS. Función

- Las funciones devuelven un único valor (tipo de datos), los procedimientos pueden devolver 0,1 o N valores (tipos de datos) en forma de lista de parámetros.
- El procedimiento se declara de manera similar a la función pero su nombre no está asociado al valor que devuelve.
- Tanto en las funciones como en los procedimientos los parámetros actuales deben coincidir en tipo, número y orden con los parámetros formales.

```
Procedure GuardarFecha (dia:string;m:integer;año:integer);  
Begin  
...  
End;
```

```
GuardarFecha('Lunes',10,2010)
```

Pasaje de parámetros



Por **COPIA**.

Son parámetros unidireccionales que se utilizan para proporcionar información a un procedimiento.

Estos no devuelven valores.

```
procedure PorCopia (numero:integer);  
Begin  
Numero:= 2000;  
Writeln(numero);  
End;
```

```
Num:=10;  
PorCopia(num);  
Writeln(num);  
  
//Resultado  
2000  
10
```



Por **REFERENCIA**. Son parámetros que se utilizan tanto para transmitir como para recibir valores entre el procedimiento y el programa/subprograma llamador. Pueden actuar como parámetros de entrada o entrada/salida.

```
procedure PorReferencia (var numero:integer);  
Begin  
Numero:= 2000;  
Writeln(numero);  
End;
```

```
Num:=10;  
PorCopia(num);  
Writeln(num);  
  
//Resultado  
2000  
2000
```

Pasaje de parámetros

Algunas reglas para utilizar parámetros por copia o por referencia:

- **Si la información que se envía al procedimiento no tiene que ser devuelta al programa llamador estamos en presencia de un pasaje por copia o valor (entrada).**
- **Si se tiene que devolver información al programa llamador, el pasaje se realiza por referencia (salidas).**
- **Si la información que se envía al procedimiento puede ser modificada, el pasaje se realiza por referencia (entrada/salida).**

Ámbito de las variables

Variables Local

Una variable local esta declarada dentro de un subprograma y se dice que su ámbito es el del subprograma. Es decir, esta variable solo esta disponible durante la ejecución del mismo y pierde su valor luego de su finalización.

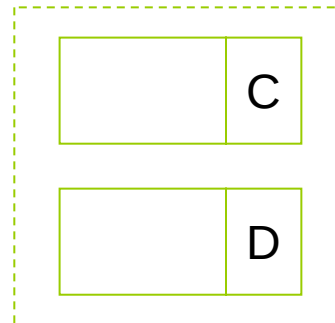
Variables Global

Una variable global esta declarada en el programa principal. Al contrario de las locales pueden ser utilizadas dentro del programa principal como dentro de todos los subprogramas.

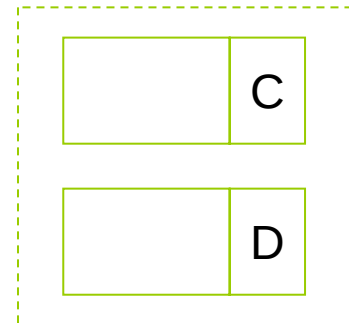
Ámbito de las variables

```
Program Ambito;  
Var  
  A:integer;  
  B:integer;  
  Procedure Demo;  
    var  
      C:integer;  
      D:integer;  
    begin  
      C:=10;  
      D:=20;  
      A:=50;  
      B:= 500;  
    end;  
  Begin  
    A:=100;  
    B:=200;  
    writeln(A);  
    writeln(B);  
    Demo;  
    Writeln(A);  
    Writeln(B);  
Writeln(C);  
Writeln(D);  
  End.
```

Ambito



Demo



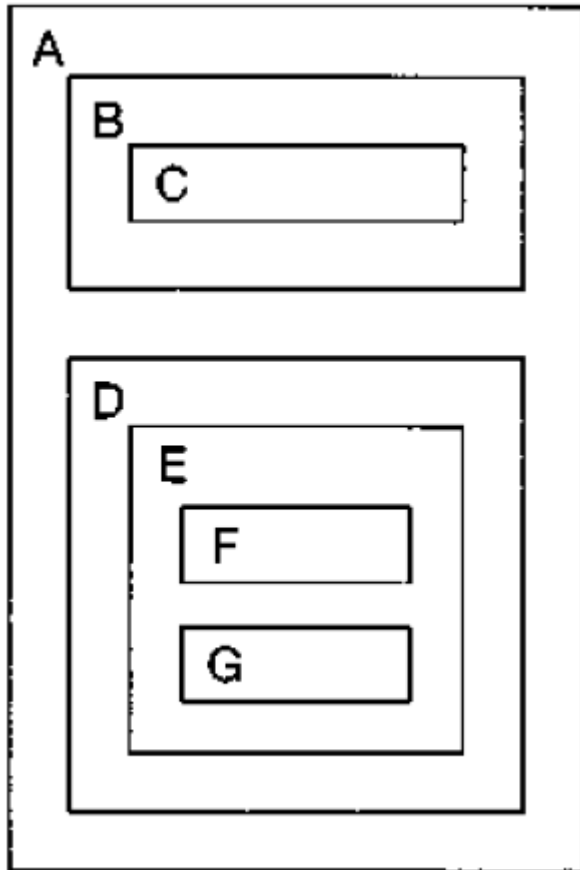
Ámbito de las variables

A tener en cuenta...

- **El uso de variables locales hace a los subprogramas independientes.**
- **Si varios programadores están implicados en un desarrollo pueden trabajar de forma independiente sobre el mismo proyecto.**
- **Una variable local en un subprograma no tiene ningún significado en otro subprograma.**
- **Una variable global puede ser de utilidad en un pequeño programa para comunicar algunos procedimientos. Sin embargo se complica su uso en procedimientos anidados o programas con complejidad.**

Ámbito de las variables

Accesibilidad de las variables y procedimientos anidados



Variables en	Accesibles desde..
A	A,B,C,D,E,F,G
B	B,C
C	C
D	D,E,F,G
E	E,F,G
F	F
G	G